

[Articles](#) · Analysis

Laya: The Open-Source Jev Alternative, Benchmarked Honestly

Laya is the Apache 2.0, 421M-parameter open-weights answer to TypeSafe's Jev: 32.8 ms decisions, 100+ languages, \$0 per token. We tested it on a CPU VPS and separate the real wins from the fine-print limits.

**AJ Awan**

21 September 2026 · 13 min read



LinkedIn



X Post



Copy link

OPEN-SOURCE AI DECISION MODELS

Laya: The Open-Source Jev Alternative

Tested first-hand. Benchmarked honestly. Apache 2.0.

32.8 ms

decision latency on a T4 GPU
7.2 ms per question batched

\$0

per million tokens, self-hosted
vs Jev metered at \$0.042

421M

open weights, Apache 2.0
3 checkpoints, 100+ languages

flowtivity.ai research note, September 2026

On this page

Last Updated: September 21, 2026

Laya is an open-source decision model family from Convai Innovations that answers typed questions instead of generating text, released under Apache 2.0 on September 18, 2026, three days after TypeSafe AI launched Jev. It returns choice, score and boolean answers with calibrated probabilities in 32.8 milliseconds on a T4 GPU, covers 100+ languages through automatic checkpoint routing, and charges nothing per token because you host it yourself. Against Jev's \$0.042 per million input tokens and 236 to 276 millisecond latency, that reads like an easy win. It is not. The headline 0.766 accuracy belongs to a checkpoint fine-tuned on the benchmark's own training split, while the zero-shot base model scores 0.362, below the 0.461 majority-class baseline. We installed Laya on a CPU-only VPS and measured it ourselves, and this guide separates what the numbers prove from what they only suggest.

What is Laya?

Laya is a non-autoregressive System 1 decision engine. You send a state, meaning any text, email, ticket or JSON document, plus typed questions whose answer spaces you define, and Laya evaluates every question in a single forward pass, returning typed answers with probability distributions instead of prose. Three checkpoints ship under Apache 2.0 on Hugging Face, installable with `pip install laya`, and a built-in router picks the right checkpoint per request. It is the open-weights answer to Jev, TypeSafe AI's closed decision model that launched September 15, 2026 after two years in stealth.

The origin story is unusual enough to matter. Jev's core idea, scoring typed answers in parallel instead of generating text token by token, is one

that independent researcher Nandakishor Mukkunoth says he published first: a March 2025 arXiv paper on reinforcement-learned sales conversion decisions (arXiv:2503.23303) and a September 2025 follow-up (arXiv:2510.01237) formalising schema-based decisions guided by reinforcement learning. TypeSafe launched Jev without technical papers, open weights or training datasets. "Instead of staying bitter, I decided to take everything I learned, fix every architectural limitation of the old approach, and build a completely open, horizontal System 1 decision model family," says Nandakishor Mukkunoth, founder of Convai Innovations. One honesty note from wavect.io's review: the three days between the two launches separate publication dates, not verified development timelines, so treat the David-and-Goliath framing as marketing-adjacent. The code and weights being downloadable is the part that is simply true.



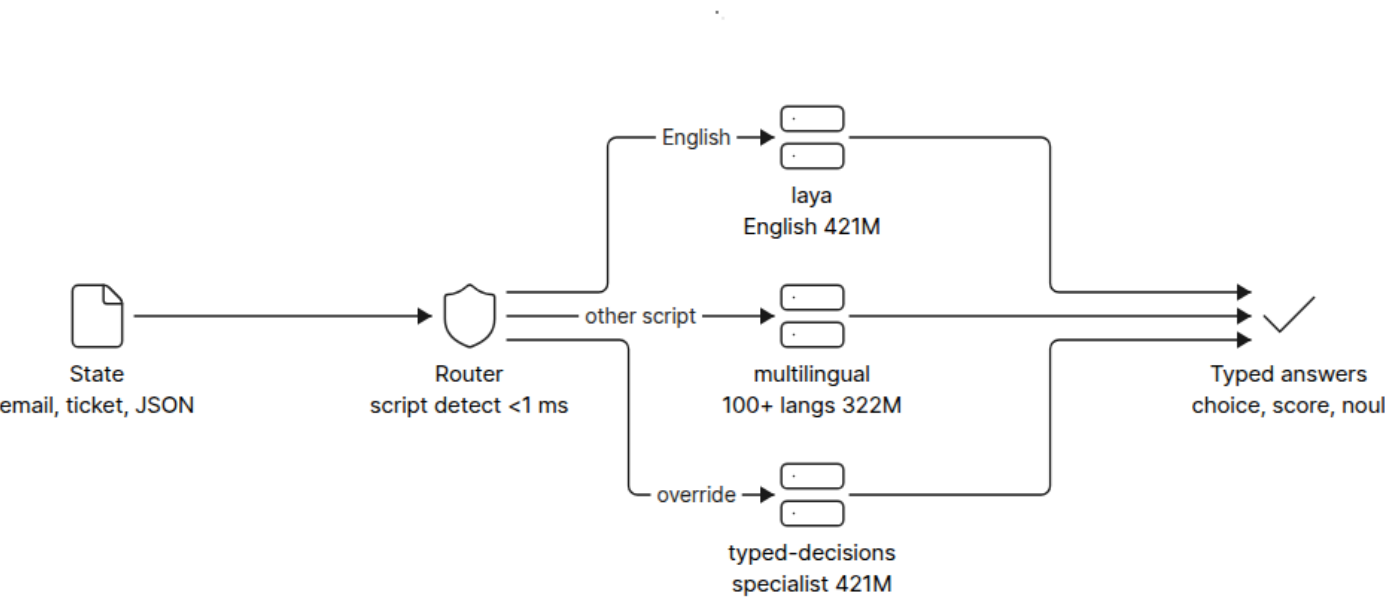
At a glance: 18 months from arXiv preprint to open-weights release, with Jev's closed launch landing three days before Laya shipped.

How does Laya work?

Laya evaluates three question primitives against any state in one forward pass. Choice picks one option from a dictionary you define and returns the full probability distribution across options plus a confidence value. Score places the item on an ordinal rubric, such as a 0 to 3 urgency scale. Noul asks a direct yes-or-no question and returns a calibrated probability

P(true). Because the output space is purely numbers and probabilities, the model physically cannot hallucinate prose or emit malformed JSON.

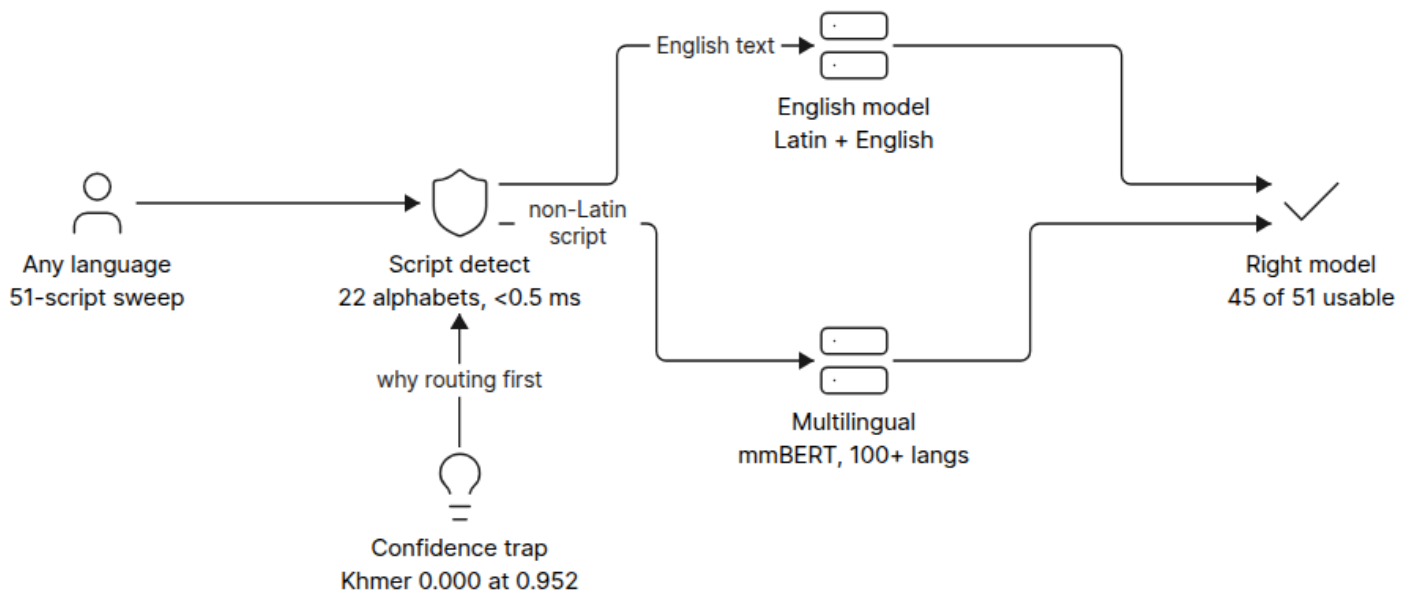
Under the hood this is a small, sensible stack rather than a new architecture: ModernBERT-large supplies 395 million of the English checkpoint's 421 million parameters, topped with a decision head trained from scratch, two transformer layers, an option scorer and an act-versus-escalate gate. Training uses RLCD, reinforcement learning against strictly proper scoring rules, which is what makes the probabilities worth branching production code on.



How it works: Laya routes any state to one of three Apache 2.0 checkpoints in under a millisecond, then returns typed answers in a single forward pass.

Checkpoint	Backbone	Params	Context	Built for
laya	ModernBERT-large	421M	512 tokens	English triage, guardrails, email
laya-multilingual	mmBERT-base, 256k vocab	322M	1024 tokens	100+ languages, 2.2x faster
laya-typed-decisions	ModernBERT-large	421M	1024 tokens	specialist decision workflows

The router deserves its own paragraph because it fixes a failure mode most teams would ship blind. On a 51-language MASSIVE sweep, the English checkpoint scored 0.000 accuracy on Khmer while reporting 0.952 confidence, and its mean confidence never dropped below 0.885 even when accuracy was zero. A confident model that cannot read the script is a silent production failure. Laya's router detects the Unicode script across 22 alphabets in under half a millisecond, before the forward pass, and dispatches to the multilingual checkpoint when needed. With Router(preload=True), all checkpoints stay resident in memory, eliminating a measured 7 to 10 second cold-swap penalty when traffic alternates languages.



How it works: script detection happens before the forward pass, because a confident English model on Khmer scores 0.000 accuracy at 0.952 confidence.

Laya vs Jev: what the benchmarks actually say

According to Laya's published benchmark report, the routed stack beats Jev 1.13.0 on hard-label accuracy (0.766 vs 0.727 on 2,000 typed decisions), AG News classification (0.950 vs 0.910), calibration error after temperature refitting (0.081 vs 0.246 ECE) and latency (32.8 ms vs 236 to 276 ms, roughly 7.8x). Jev's numbers, the report itself notes, are third-party published and were never measured in the same run. According to

wavect.io's independent review, there are also metrics that favour Jev: soft accuracy of 0.580 vs Laya's 0.471, and calibration on the published evaluation of 0.144 vs Laya specialist's 0.213. Two defensible readings exist, which is exactly why we ran our own test below.

Dimension	Laya (open)	Jev 1.13.0 (closed)
Latency, 1 question	32.8 ms on T4, self-measured	236 to 276 ms, third-party published
Latency, 10 questions batched	72.3 ms total (7.2 ms each)	about 1,500 ms serial
Typed-decisions accuracy	0.766 fine-tuned, 0.362 zero-shot	0.727 published, zero-shot not stated
Calibration (ECE)	0.081 after per-question temperature refit	0.246 reported
Languages above 3x random	45 of 51 evaluated	no published benchmark
Cost per 1M input tokens	\$0, self-hosted	\$0.042 metered API
Weights and audit access	full safetensors, Apache 2.0	none, waitlist API
Wide option sets	0.425 on Banking77 (77 labels)	0.870 on Banking77

Laya vs Jev: the scoreboard

Laya figures self-measured on T4. Jev figures third-party published, never measured in the same run.

	LAYA (Apache 2.0)	vs	JEV 1.13.0 (closed)
Latency, 1 question	32.8 ms		236 to 276 ms
Typed accuracy*	0.766		0.727
Zero-shot accuracy	0.362, fine-tune it		not published
Calibration (ECE)	0.081 after refit		0.246 reported
Cost per 1M tokens	\$0 self-hosted		\$0.042 metered
Languages, 3x random	45 of 51		no benchmark
Wide option sets	0.425 on 77 labels		0.870 on 77 labels
Weights and audit access	full safetensors		none, API only

*Laya fine-tuned on the benchmark train split; zero-shot is 0.362

At a glance: the scoreboard with its asterisks. Laya leads on speed, cost and openness, Jev leads on wide option sets, and the accuracy story depends on fine-tuning.

"Laya is a fast base to specialise, not a zero-shot decision engine," the Hugging Face model card states plainly. That single sentence is the difference between a good deployment and a disappointing one, because the gap between 0.362 zero-shot and 0.766 fine-tuned is entirely the training split of one benchmark.

We tested it: Laya on a CPU VPS

We installed Laya 0.3.4 from PyPI on a 4 vCPU, 7 GB RAM VPS with no GPU, downloaded the roughly 808 MB English checkpoint, and ran the project's own support-ticket quickstart: one state, three questions, measured end to end. Import took 26 seconds, checkpoint load took 675 seconds including the download, the first cold predict took 85.4 seconds, and warm predicts settled at a 49.4 second median across five runs, against a documented 193 to 464 millisecond CPU figure on healthier hardware. Our box is the floor of the floor, but the deployment lesson is

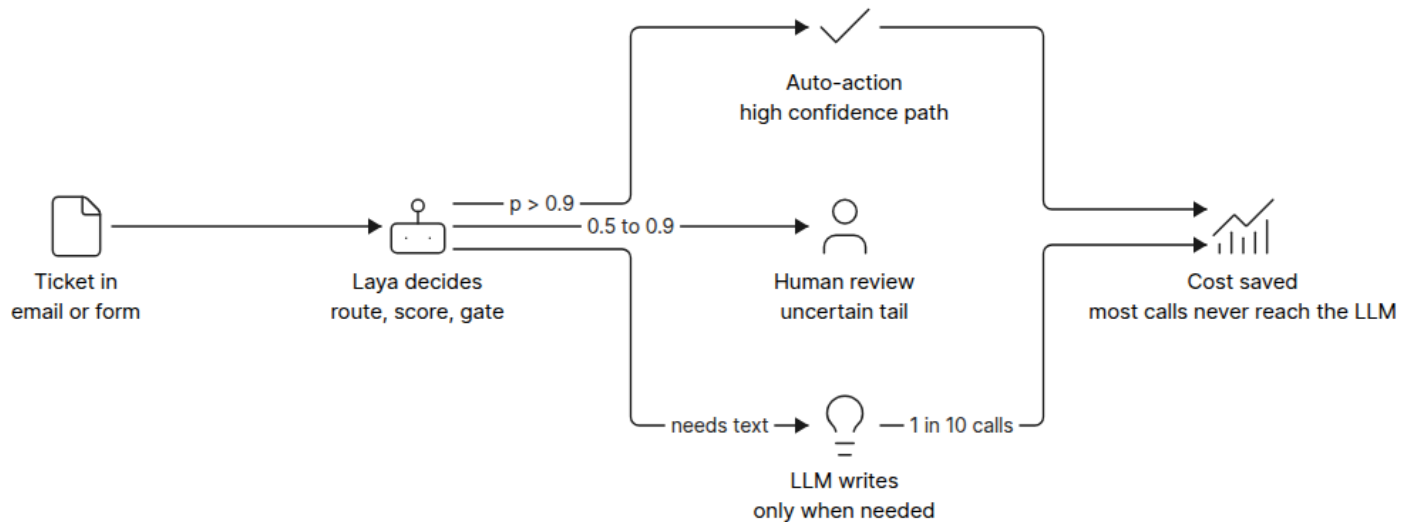
real: without a GPU, Laya is a batch tool, not a request path, so budget a T4-class GPU for anything interactive.

The decision quality on our test ticket, a production outage with a refund demand, was sensible: queue billing at 0.532 confidence, urgency 2.73 of 3, churn risk 19.3 percent. Notably, the published quickstart example for the same ticket shows infrastructure at 0.96 confidence and churn risk at 91.4 percent, so current-release outputs differ from the docs. We report that without drama: it is the strongest argument for benchmarking any decision model, open or closed, on your own traffic before a single threshold ships to production, and it is exactly what open weights make possible, because you can inspect, refit and verify everything yourself.

Where Laya wins today

Narrow, high-volume binary and few-option decisions are Laya's home ground, and the published workflow numbers back it up: 0.993 accuracy on Enron email spam filtering, 0.980 on phishing detection, and 0.755 to 0.762 on held-out jailbreak and guardrail detection, rising to 0.931 accuracy at 50 percent selective coverage when low-confidence cases defer. Add 45 of 51 usable languages, air-gapped deployment and a \$0 token bill, and the economics of the decision layer change completely: a 421M model on hardware you already own, inside the request path, where a frontier API call cannot fit.

For the automation stacks we build at Flowtivity for growing Australian businesses, three shapes fit immediately. Support ticket routing with urgency scoring, because after-hours triage for trades and allied health is exactly a 4-way choice plus a rubric. Email and invoice triage with phishing gating, which is the 0.98-plus territory. And LLM guardrails in front of client-facing agents, where a 30-millisecond gate on every prompt beats a post-hoc moderation scan. There is also a sovereignty angle that matters locally: weights on your own hardware keep customer data inside your perimeter, which simplifies conversations in health and finance.

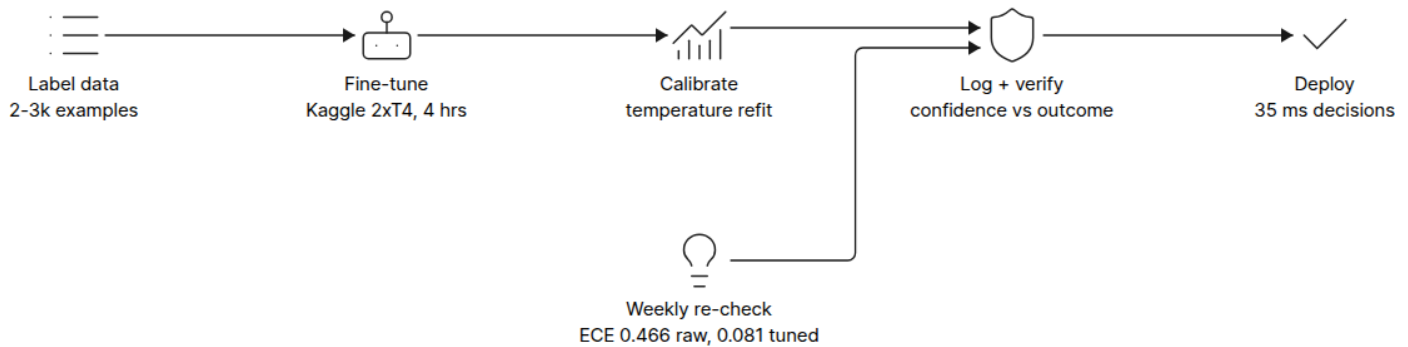


How it works: Laya decides route, score and gate in milliseconds, so only the uncertain tail reaches humans and only true writing tasks reach the LLM.

Where Laya loses: the honest limits

Four limits, all documented by the project itself. First, zero-shot weakness: base models score about 0.35 on the typed-decisions benchmark against a 0.318 random baseline, so plan to fine-tune on a few thousand labelled examples of your own decision. Second, option count: choice questions degrade past 20 options, scoring 0.425 on Banking77's 77 labels where Jev scores 0.870, because options share a fixed token budget. Third, calibration is work, not a property: raw expected calibration error ships at 0.466 and reaches the advertised 0.081 only after fitting a temperature scalar per question type on your own distribution. Fourth, context: the English checkpoint's 512-token window is roughly one email, not a support thread, so long histories need the multilingual or specialist checkpoints or chunking.

The mitigation path is refreshingly cheap. The project ships a Kaggle notebook that fine-tunes the typed-decisions checkpoint on two free T4 GPUs in about four hours, and the SDK exposes selective-subfolder downloads so an 808 MB English deployment never pulls the full 2.5 GB bundle. Budget the labelling and calibration days before trusting any confidence number, then log confidence against outcomes weekly.



How it works: the path from zero-shot 0.362 to specialist 0.766 runs through your labels, a free Kaggle fine-tune and a per-question temperature refit.

Who should run Laya, and who should wait

Run Laya if you can label a few thousand examples, need data on your own hardware, want sub-50-millisecond decisions inside a request path, or serve multilingual traffic without per-token bills. Wait, or stay on Jev, if you need zero setup today, broad zero-shot coverage out of the box, option sets wider than 20, or multi-step reasoning inside a single call. Both tools lose when asked to write anything, so the LLM stays either way.

The cost math is worth writing down. Our published Jev cost model put a 40,000-decisions-per-month intake workflow at \$1.34 a month on metered pricing, already trivial against roughly \$900 for the frontier-model route. Self-hosting Laya zeroes even that API bill if a T4-class GPU is already in the stack next to your LLM serving, which is the common case. At these prices the decision is operational, not financial: metered convenience with a waitlist, versus owned infrastructure with fine-tuning responsibility.

The lesson: a model is not a moat

Three days after a funded frontier lab launched a closed decision model, an independent researcher shipped an open one with weights, code, datasets and benchmark harnesses. Whether or not the priority dispute ever resolves, the market signal already has. "A model lead is not a business moat," writes Kevin Riedl in wavect.io's review of the episode.

The defensible layers are your data, your thresholds, your audit trail and the workflow wrapped around whichever model you chose this quarter.

That is how we now architect decision layers for clients: behind a thin interface, so Jev, Laya or the inevitable next release can be swapped in a day of work. This is the third piece in our series on the decision-model shift, after the original [Jev teardown](#) and the [Jev use cases field guide](#). The pattern across all three: the models keep changing, the shape of the win, decide cheap, gate hard, write rarely, does not.

AI

Open Source

Decision Models

Laya

Jev

LLM Engineering

**Written by AJ Awan**

Founder of Flowtivity, based on the Gold Coast. AJ brings enterprise technology experience to practical AI work for Australian businesses.

[More about AJ](#)

One email a month, no noise

Practical AI notes for Australian businesses. Unsubscribe anytime.

Email address

✉ you@example.com

Subscribe

Keep reading

FLOWTIVITY ANALYSIS

Claude Opens Custom Plugin Uploads

What Anthropic's plugin directory portal means for builders and growing teams

Sep 25

2026 launch date of the
directory submission portal

2 paths

Single MCP connector or
a bundled plugin on GitHub

2.0

MCP spec supported, with
Apps UI and managed OAuth

flowtivity.ai/blog | Last Updated: September 27, 2026

Claude Custom Plugin Uploads: What Anthropic's New Plugin Directory Portal Means

Anthropic opened Claude plugin submissions and private plugin uploads on September 25, 2026. Here is what plugins are, how to build one, and...

27 September 2026

FLOWTIVITY DECISION GUIDE

Laya or Jev: Which Should You Run?

One Gateway. Every Model.

Spend caps for your AI agents, cloud and local

1

OpenAI-compatible API
for all your agents

1000+

models reachable
hosted, BYOK, local

0%

markup on your keys
Apache-2.0 open source

A first look at Experiential, the open source AI gateway (5,600+ GitHub stars)

Can One AI Gateway Control What Your Agents Are Allowed to Spend?

Experiential is an open source AI gateway that puts cloud, BYOK and local models behind one OpenAI-compatible API with per-agent spend caps...

23 September 2026

ONE GOOD PLACE TO START

What would you like to take off your plate?

Bring a process that feels repetitive or harder than it needs to be. We'll help you find a practical first step.

Book a free consult →

A free 1-hour conversation with AJ. No pressure, no pitch.